Algorithms

3

Chapter Summary

- **Algorithms**
  - **Example Algorithms**
- **Growth of Functions**
  - **Big-*O* and other Notation**
- **Complexity of Algorithms**

Problems and Algorithms

- In many domains there are key general problems that ask for output with specific properties when given valid input.
- The first step is to precisely state the problem, using the appropriate structures to specify the input and the desired output.
- We then solve the general problem by specifying the steps of a procedure that takes a valid input and produces the desired output. This procedure is called an *algorithm*.

Algorithms

- An algorithm is a finite set of precise instructions for performing a computation or for solving a problem.

- Example: Finding the Maximum Element in a Finite Sequence

  procedure *max*(a_1,a_2,…,a_n:integers)

  max:=a_1

  for j:= 2 to n

  if *max* < a_j then max:= a_j

  {*max* is the largest element}

Specifying Algorithms

- Algorithms can be specified in different ways. Their steps can be described in English or in *pseudocode.*
- Pseudocode is an intermediate step between an English language description of the steps and a coding of these steps using a programming language.
- The form of pseudocode we use is specified in Appendix 3. It uses some of the structures found in popular languages such as C++ and Java.
- Programmers can use the description of an algorithm in pseudocode to construct a program in a particular language.
- Pseudocode helps us analyze the time required to solve a problem using an algorithm, independent of the actual programming language used to implement algorithm.

Properties of algorithm

- *Input*: An algorithm has input values from a specified set.
- *Output*: From the input values, the algorithm produces the output values from a specified set. The output values are the solution.
- *Correctness*: An algorithm should produce the correct output values for each set of input values.
- *Finiteness*: An algorithm should produce the output after a finite number of steps for any input.
- *Effectiveness*: It must be possible to perform each step of the algorithm correctly and in a finite amount of time.
- *Generality*: The algorithm should work for all problems of the desired form.

Some Example Algorithm Problems

- Four classes of problems will be studied
  1. *Searching*: finding the position of a particular element in a list.
  2. *Sorting*: putting the elements of a list into increasing order.
  3. *String Matching*: Finding where a pattern occurs in a text string is
  4. *Optimization Problems*: determining the optimal value (maximum or minimum) of a particular quantity over all possible inputs

Searching Algorithms
(The linear search algorithm)

- **Procedure** linear search (x: integer, a_1, a_2…, a_n : distinct integers)
- i : = 1
- **While** (i≦n and x ≠ a_i)
- i: = i + 1
- If i ≦ n **then** location : = i
- **Else** location : = 0
- {location is the subscript of term that equals x ,or is 0 if x is not found}

Searching Algorithms (The Binary Search Algorithm)

- **Procedure** binary search(x:integer, a_1, a_2…, a_n : increasing int)
- i : =1{i is left endpoint of search interval }
- j : =n {j is right endpoint of search interval }
- **While** i < j
- **Begin**
- m : =[(i + j)/2]
- If x > a_m then i : =m + 1
- Else j: = m
- End
- If x = a_i **then** location : = i
- **Else** location : = 0
- {location is the subscript of term equal to x ,or 0 if x is not found }

Binary Search

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8. The value in the 8th position is 10. Since 19 > 10, further search is restricted to positions 9 through 16.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

2. The midpoint of the list (positions 9 through 16) is now the 12th position with a value of 16. Since 19 > 16, further search is restricted to the 13th position and above.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

3. The midpoint of the current list is now the 14th position with a value of 19. Since 19 ≯ 19, further search is restricted to the portion from the 13th through the 14th positions .

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

4. The midpoint of the current list is now the 13th position with a value of 18. Since 19> 18, search is restricted to the portion from the 18th position through the 18th.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

5. Now the list has a single element and the loop ends. Since 19=19, the location 16 is returned.

Sorting

- To *sort* the elements of a list is to put them in increasing order (numerical order, alphabetic, and so on).
- Sorting is an important problem because:
  - A nontrivial percentage of all computing resources are devoted to sorting different kinds of lists, especially applications involving large databases of information that need to be presented in a particular order (e.g., by customer, part number etc.).
  - An amazing number of fundamentally different algorithms have been invented for sorting. Their relative advantages and disadvantages have been studied extensively.
  - Sorting algorithms are useful to illustrate the basic notions of computer science.
- A variety of sorting algorithms are studied in this book; binary, insertion, bubble, selection, merge, quick, and tournament.

Bubble Sort

- *Bubble sort* makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.

procedure *bubblesort*($a_1,\ldots,a_n$: real numbers with $n \geq 2$)

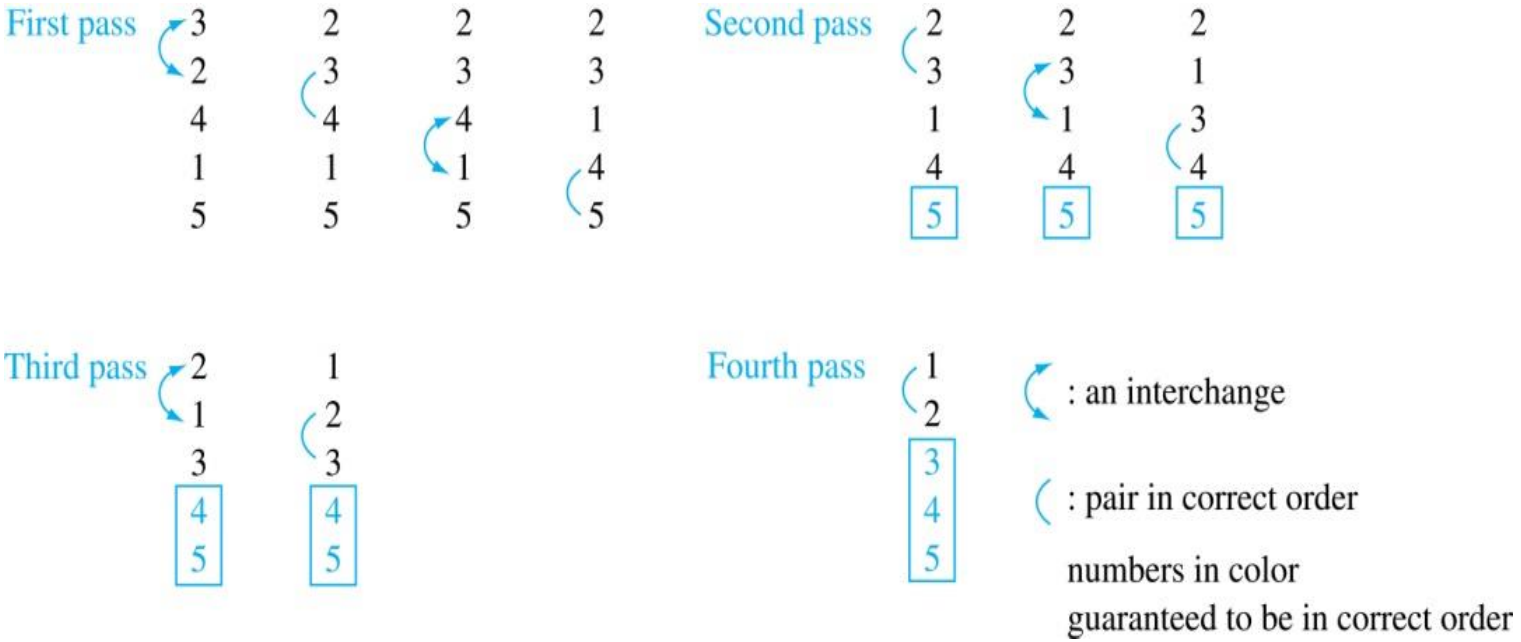
  for i := 1 to $n-1$

    for j := 1 to $n-i$

      if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}

{$a_1,\ldots, a_n$ is now in increasing order}

Example: Show the steps of bubble sort with 3 2 4 1 5



- At the first pass the largest element has been put into the correct position
- At the end of the second pass, the 2nd largest element has been put into the correct position.
- In each subsequent pass, an additional element is put in the correct position.

Insertion Sort

- *Insertion sort* begins with the 2nd element. It compares the 2nd element with the 1st and puts it before the first if it is not larger.
- Next the 3rd element is put into the correct position among the first 3 elements.
- In each subsequent pass, the n+1st element is put into its correct position among the first n+1 elements.
- Linear search is used to find the correct position.

procedure *insertion sort*
($a_1,\ldots,a_n$:
real numbers with $n \geq 2$)
for $j := 2$ to n
$i := 1$
while $a_j > a_i$
$i := i + 1$
$m := a_j$
for $k := 0$ to $j - i - 1$
$a_{j-k} := a_{j-k-1}$
$a_i := m$
{Now $a_1,\ldots,a_n$ is in increasing order}

Example: Show all the steps of insertion sort with the input: 3 2 4 1 5

i. 2 3 4 1 5 (*first two positions are interchanged*)

ii. 2 3 4 1 5 (*third element remains in its position*)

iii. 1 2 3 4 5 (*fourth is placed at beginning*)

iv. 1 2 3 4 5 (*fifth element remains in its position*)

Quick Sorting (self-study)

Quick Sort is based on the concept of **Divide and Conquer**

```
void quickSort(int arr[], int left, int right) {
      int i = left, j = right;
      int tmp;
      int pivot = arr[(left + right) / 2];
       /* partition */
      while (i <= j) {
            while (arr[i] < pivot)
                  i++;
            while (arr[j] > pivot)
                  j--;

      if (j <= j]) {
            tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
            i++;
            j--;
        }
    };
    /* recursion */
    if (left < j)
          quickSort(arr, left, j);
    if (i < right)
          quickSort(arr, i, right);}
```

String Matching

- **Self-study**

Greedy Algorithm

- *Optimization problems* minimize or maximize some parameter over all possible inputs.
- Among the many optimization problems we will study are:
  - Finding a route between two cities with the smallest total mileage.
  - Determining how to encode messages using the fewest possible bits.
  - Finding the fiber links between network nodes using the least amount of fiber.
- Optimization problems can often be solved using a *greedy algorithm*, which makes the “best” choice at each step. Making the “best choice” at each step does not necessarily produce an optimal solution to the overall problem, but in many instances, it does.
- After specifying what the “best choice” at each step is, we try to prove that this approach always produces an optimal solution, or find a counterexample to show that it does not.

Greedy Algorithms: Making Change

Example: Design a greedy algorithm for making change (in U.S. money) of *n* cents with the following coins: quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cent) , using the least total number of coins.

Idea: At each step choose the coin with the largest possible value that does not exceed the amount of change left.

1. **If $n = 67$ cents, first choose a quarter leaving $67-25=42$ cents. Then choose another quarter leaving $42-25=17$ cents**
2. **Then choose 1 dime, leaving $17-10=7$ cents.**
3. **Choose 1 nickel, leaving 7 – 5 – 2 cents.**
4. **Choose a penny, leaving one cent. Choose another penny leaving 0 cents.**

Greedy Algorithms: Making Change

Solution: Greedy change-making algorithm for *n* cents. The algorithm works with any coin denominations $c_1, c_2, \ldots, c_r$.

procedure *change*($c_1, c_2, \ldots, c_r$: values of coins, where $c_1 > c_2 > \ldots > c_r$; n: a positive integer)
for $i := 1$ to r
    $d_i := 0$ [d_i counts the coins of denomination c_i]
    while $n \geq c_i$
        $d_i := d_i + 1$ [add a coin of denomination c_i]
        $n = n - c_i$
[d_i counts the coins c_i]

- **For the example of U.S. currency, we may have quarters, dimes, nickels and pennies, with $c_1 = 25$, $c_2 = 10$, $c_3 = 5$, and $c_4 = 1$.**

Proving Optimality for U.S. Coins

- Show that the change making algorithm for *U.S.* coins is optimal.

Lemma 1: If *n* is a positive integer, then *n* cents in change using quarters, dimes, nickels, and pennies, using the fewest coins possible has at most 2 dimes, at most 1 nickel, at most 4 pennies, and cannot have 2 dimes and a nickel. The total amount of change in dimes, nickels, and pennies must not exceed 24 cents.

Proof: By contradiction

- If we had 3 dimes, we could replace them with a quarter and a nickel.
- If we had 2 nickels, we could replace them with 1 dime.
- If we had 5 pennies, we could replace them with a nickel.
- If we had 2 dimes and 1 nickel, we could replace them with a quarter.
- The allowable combinations, have a maximum value of 24 cents; 2 dimes and 4 pennies.

Proving Optimality for U.S. Coins

Theorem: The greedy change-making algorithm for U.S. coins produces change using the fewest coins possible.

Proof: By contradiction.

1. Assume there is a positive integer n such that change can be made for n cents using quarters, dimes, nickels, and pennies, with a fewer total number of coins than given by the algorithm.
2. Then, $q' \leq q$ where q' is the number of quarters used in this optimal way and q is the number of quarters in the greedy algorithm's solution. But this is not possible by Lemma 1, since the value of the coins other than quarters can not be greater than 24 cents.
3. Similarly, by Lemma 1, the two algorithms must have the same number of dimes, nickels, and quarters.

Greedy Algorithms: Making Change

- Optimality depends on the denominations available.
- For U.S. coins, optimality still holds if we add half dollar coins (50 cents) and dollar coins (100 cents).
- But if we allow only quarters (25 cents), dimes (10 cents), and pennies (1 cent), the algorithm no longer produces the minimum number of coins.
  - Consider the example of 31 cents. The optimal number of coins is 4, i.e., 3 dimes and 1 penny. What does the algorithm output?

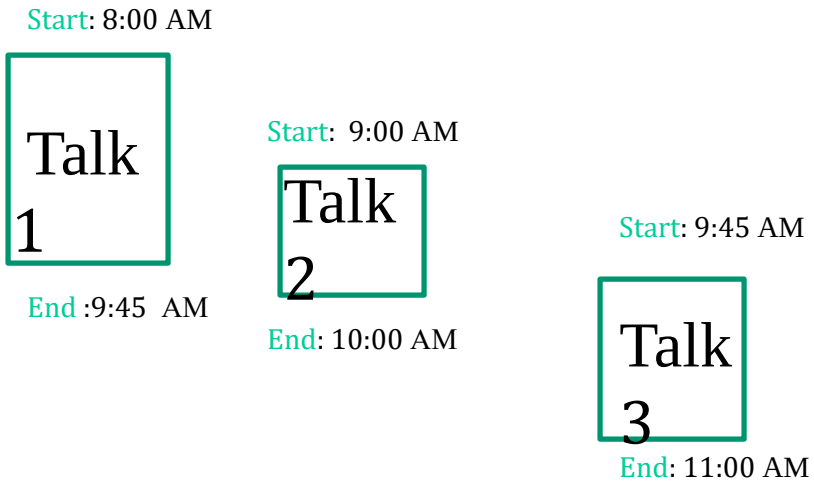
Greedy Scheduling

Example: We have a group of proposed talks with start and end times. Construct a greedy algorithm to schedule as many as possible in a lecture hall, under the following assumptions:

- When a talk starts, it continues till the end.
- No two talks can occur at the same time.
- A talk can begin at the same time that another ends.
- Once we have selected some of the talks, we cannot add a talk which is incompatible with those already selected because it overlaps at least one of these previously selected talks.
- How should we make the “best choice” at each step of the algorithm? That is, which talk do we pick ?
  - The talk that starts earliest among those compatible with already chosen talks?
  - The talk that is shortest among those already compatible?
  - The talk that ends earliest among those compatible with already chosen talks?

Greedy Scheduling

- Picking the shortest talk doesn't work.



- Can you find a counterexample here?
- But picking the one that ends soonest does work. The algorithm is specified on the next page.

Greedy Scheduling

Solution: At each step, choose the talks with the earliest ending time among the talks compatible with those selected.

procedure *schedule*($s_1 \le s_2 \le \ldots \le s_n$: start times, $e_1 \le e_2 \le \ldots \le e_n$: end times)
sort talks by finish time and reorder so that $e_1 \le e_2 \le \ldots \le e_n$
$S := \emptyset$
for $j := 1$ to n
  if talk j is compatible with S then
    S := S ∅∪{talk j}
return S [S is the set of talks scheduled]

- Will be proven correct by induction in Chapter 5.

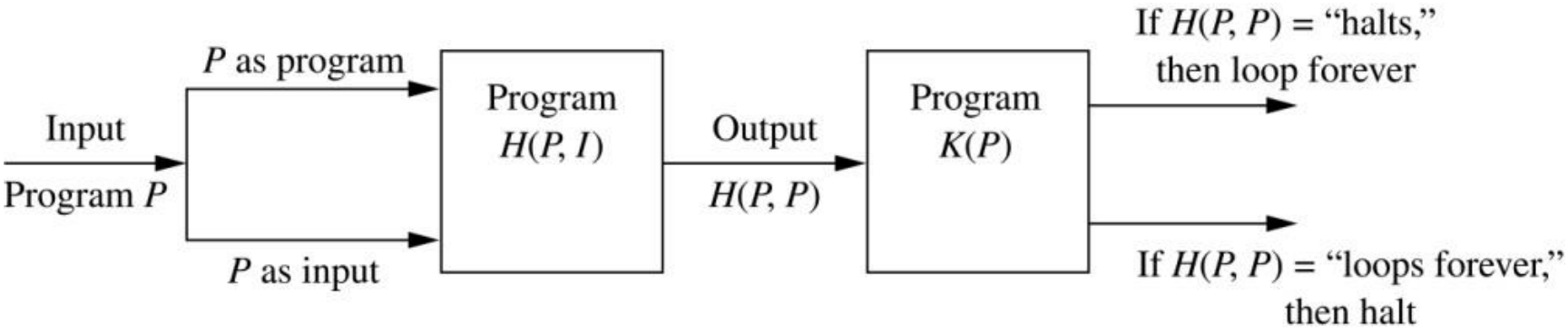
* The Halting Problems

Example: Can we develop a procedure that takes as input a computer program along with its input and determines whether the program will eventually halt with that input.

- **Solution: Proof by contradiction.**
- **Assume that there is such a procedure and call it *H*(*P*,*I*). The procedure *H*(*P*,*I*) takes as input a program *P* and the input *I* to *P*.**
  - **H outputs "halt" if it is the case that *P* will stop when run with input *I*.**
  - **Otherwise, *H* outputs "loops forever."**

* The Halting Problems

- **Since a program is a string of characters, we can call *H*(*P*,*P*). Construct a procedure *K*(*P*), which works as follows.**
  - **If *H*(*P*,*P*) outputs "loops forever" then *K*(*P*) halts.**
  - **If *H*(*P*,*P*) outputs "halt" then *K*(*P*) goes into an infinite loop printing "ha" on each iteration.**



* The Halting Problems

- **Now we call *K* with *K* as input, i.e. *K*(*K*).**
  - **If the output of *H*(*K*,*K*) is "loops forever" then *K*(*K*) halts. A Contradiction.**
  - **If the output of *H*(*K*,*K*) is "halts" then *K*(*K*) loops forever. A Contradiction.**
- **Therefore, there can not be a procedure that can decide whether or not an arbitrary program halts. The halting problem is unsolvable.**

Exercises

- **P213: 6, 20, 47, 60**

3.2 The growth of functions

Factors to be considered for choosing an algorithm

- Simplicity - Easy to implement and obviously correct
- Clarity - Easy to read and to maintain
- Extensibility - Can easily be extended to new tasks
- Reusability - Can be adapted to different tasks
- *Efficiency - Uses time and space well*

Analysis

Analysis determines the running time of an algorithm based on the *input size*

- Gives you the relative rate of growth but not specific time of execution
- Allows for comparison of algorithms without implementation
- Not hardware specific
- Describes *exact* number of operations for a given input size

What is “Input Size”?

- Depends on the problem being analyzed
- The number of items in the input is often used
  - Number of values in a sorting algorithm
- May use more than one parameter
  - Number of edges and vertices in a graph

Growth of Functions

- Asymptotic efficiency of algorithms
  - How does the running time of an algorithm increase as the input increases in size without bound?
- Asymptotic notations
  - Define sets of functions that satisfy certain criteria and use these to characterize time and space complexity of algorithms

Big-*O* Notation

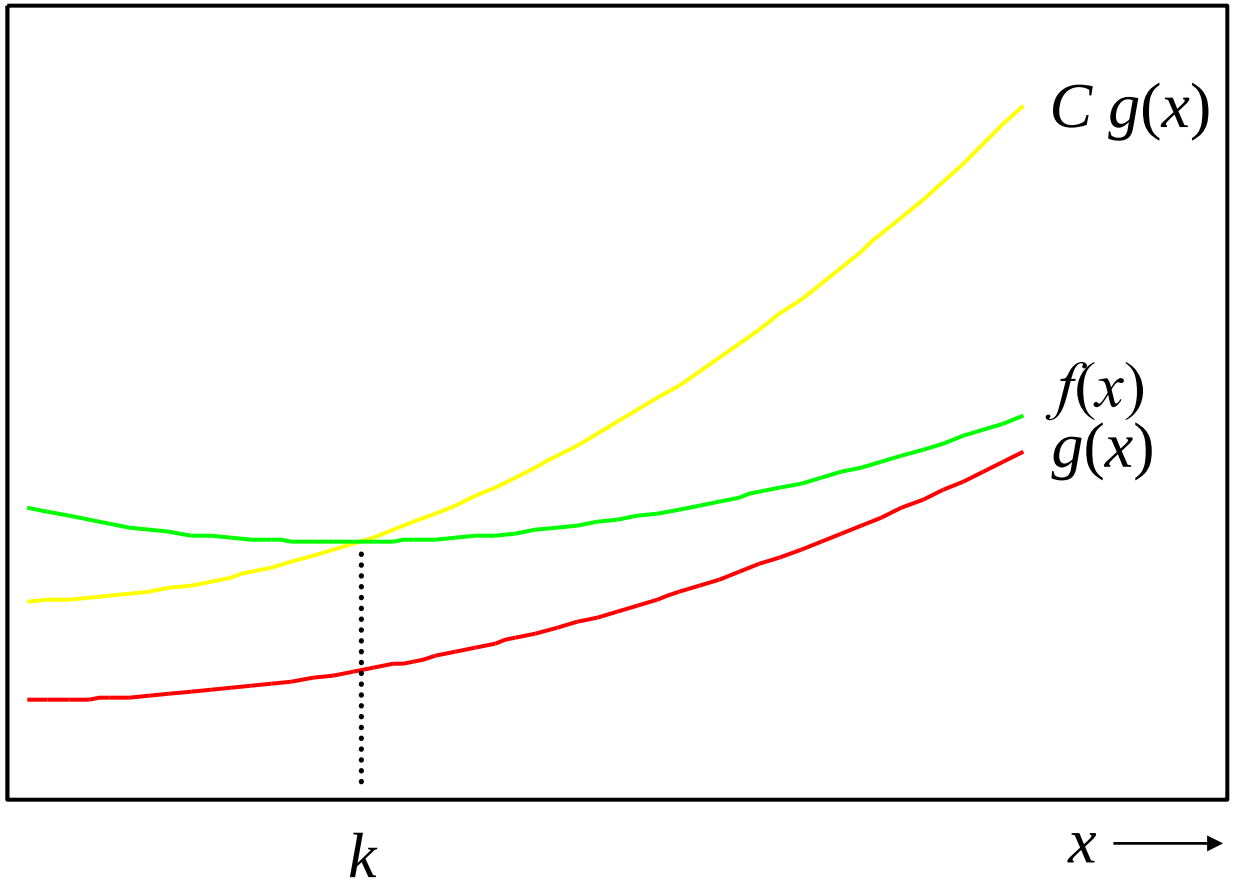
【Definition】 Let f and g be functions from N(or R) to R. We say that "$f(x)$ is $O(g(x))$" if there are positive constants C and k such that

$$|f(x)| \leq C|\, g(x)|$$

whenever $x \geq k$.

"$f(x)$ is $O(g(x))$" is read as: "$f(x)$ is big-oh of $g(x)$"

The function $f(x)$ is $O(g(x))$



Note

1. $f(x)$ is $O(g(x))$ can be denoted by $f(x) = O(g(x))$.
2. The pair C,K satisfiers the definition is never unique. Moreover, if one such pair exists, there are infinitely many such pairs.
3. When $f(x)$ is $O(g(x))$, and $h(x)$ is a function that has large absolute values than $g(x)$ does for sufficiently large values of x, it follows that $f(x)$ is $O(h(x))$.

Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.

1) $f(x) = x^2 + 2x + 1$

$\leq x^2 + 2x^2 + 1$ **For all** $x \geq 1$

$\leq x^2 + 2x^2 + x^2$ **For all** $x \geq 1$

$= 4x^2 \quad = Cx^2 \quad = Cg(x)$

We have: $C = 4, \quad k = 1, \quad g(x) = x^2$

$f(x)$ **is** $O(x^2)$

2) $0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$ **whenever** $x > 2$

3) x^2 **is** $O(x^2+2x+1)$

Note: Two functions $f(x)$ and $g(x)$ that satisfy both of these big-O relationships are *of the same order*.

Show that $7x^2$ is $O(x^3)$. Is it also true x^3 is $O(7x^2)$?

1) Note that when $x>7$, we have $7x^2 < x^3$.

Consequently, we can take $C=1$, and $k=7$, and to establish the relation $7x^2$ is $O(x^3)$.

2) $x^3 \leq C(7x^2)$

$x \leq 7C$

Note that no C exists for which $x \leq 7C$ for all $x>k$.

Some Important Big-*O* Results

【Theorem】 Let $f(x) = a_n x^n + a_{n-1}x^{n-1} + \ldots + a_1 x + a_0$, where $a_0, a_1, \ldots, a_n$ are real numbers. Then $f(x)$ is $O(x^n)$.

Proof:

Using the triangle inequality, if $x>1$ we have

$$
\begin{aligned}
|f(x)| &= |a_n x^n + a_{n-1}x^{n-1} + \ldots + a_1 x + a_0| \\
&\le |a_n| x^n + |a_{n-1}| x^{n-1} + \ldots + |a_1| x + |a_0| \\
&= x^n (|a_n| + |a_{n-1}|/x + \ldots + |a_1|/x^{n-1} + |a_0|/x^n) \\
&\le x^n (|a_n| + |a_{n-1}| + \ldots + |a_1| + |a_0|)
\end{aligned}
$$

It follows that $|f(x)| \le Cx^n$

Examples

Example: Use big-*O* notation to estimate the sum of the first *n* positive integers.

Solution: $1 + 2 + \cdots + n \leq n + n + \cdots n = n^2$

$1 + 2 + \ldots + n$ is $O(n^2)$ taking $C = 1$ and $k = 1$.

Example: Use big-*O* notation to estimate the factorial function

$$f(n) = n! = 1 \times 2 \times \cdots \times n \ .$$

Solution:

$$n! = 1 \times 2 \times \cdots \times n \leq n \times n \times \cdots \times n = n^n$$

$n!$ is $O(n^n)$ taking $C = 1$ and $k = 1$.

Examples

Example: Use big-*O* notation to estimate log *n*!

Solution: Given that $n! \leq n^n$ **(previous slide)**

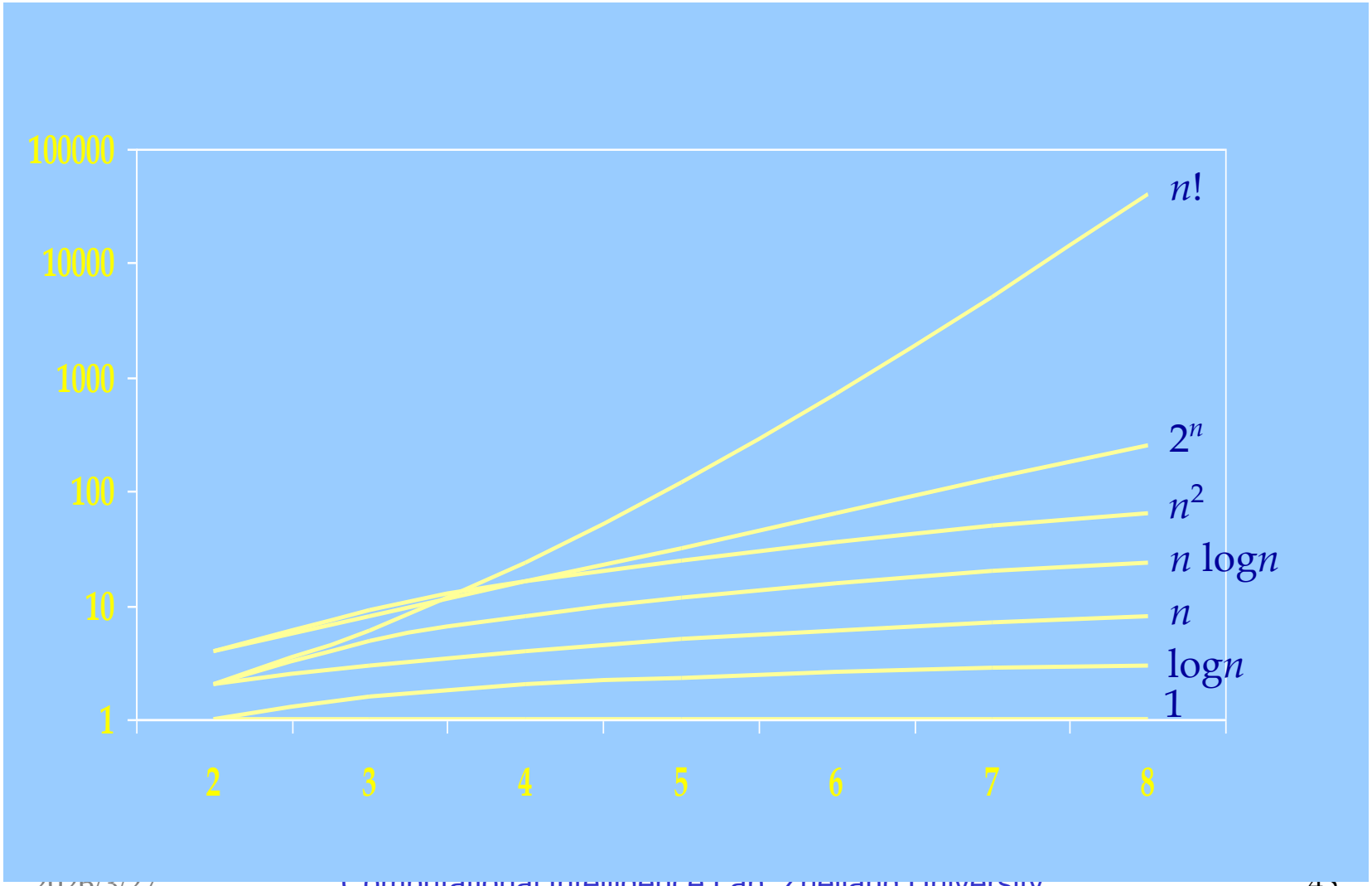
then $\log(n!) \leq n \cdot \log(n)$.

Hence, log(*n*!) is *O*(*n*·log(*n*)) taking *C* = 1 and *k* = 1.

Common Complexity Functions

Complexity	Term
$O(1)$	constant
$O(\log n)$	logarithmic
$O(n)$	linear
$O(n \log n)$	$n \log n$
$O(n^b)$	polynomial
$O(b^n)$ $b > 1$	exponential
$O(n!)$	factorial

Growth of Some Common Functions



Growth of Combinations of Functions

(1) Addition of functions

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$,

then $(f_1 + f_2)(x)$ is $O(\max(g_1(x), g_2(x)))$.

Example: What is the complexity of the function $n^2 + \log(n!)$?

Solution:

$n^2 = O(n^2)$

$\log(n!) = O(n \log n)$

Since $O(n^2) > O(n \log n)$,

$n^2 + n \log(n!) = O(n^2)$

Growth of Combinations of Functions

(2) Multiplication of functions

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$,

then $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$.

Example: What is the complexity of the function $3n\log(n!)$?

Solution:

$3n = O(n)$

$\log(n!) = O(n \log n)$

$3n\log(n!) = O(n \times n \log n) = O(n^2 \log n)$

Big-Omega

- $f(x)=O(g(x))$ only provides an upper bound in terms of $g(x)$ for $f(x)$.
- What do we do for a lower bound?
- Big-Omega provides a lower bound for a function to within a constant factor.

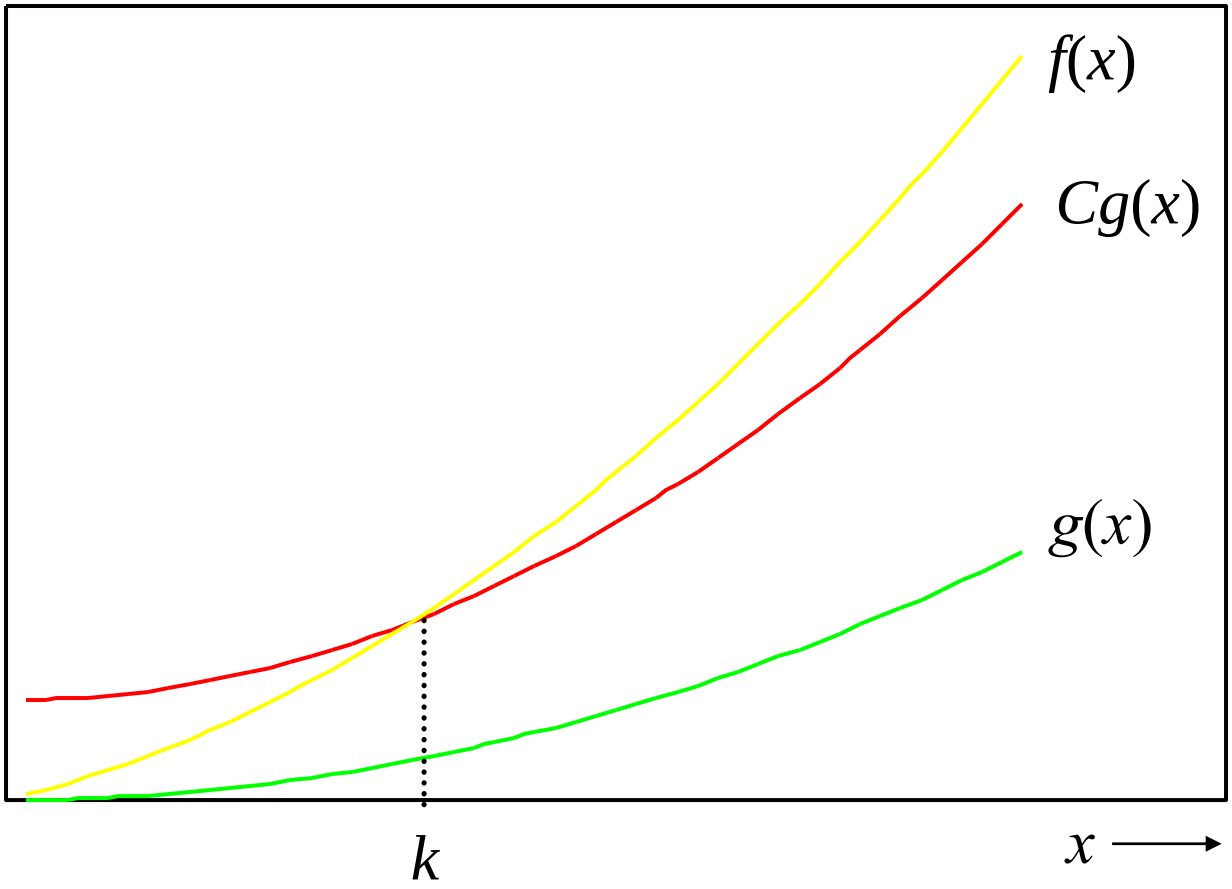
Definition: Let f and g be functions from N (or R) to R. We say that "$f(x)$ is $\Omega(g(x))$" if there are positive constants C and k such that

$$|f(x)| \geq C|g(x)|$$

whenever $x > k$.

$f(x)$ is $\Omega(g(x))$ is read as: "$f(x)$ is big-Omega of $g(x)$".

Big-Omega



Example

Show that $f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(x^3)$.

Solution:

$$f(x) = 8x^3 + 5x^2 + 7$$

$$\geq 8x^3 \qquad \text{For all } x \geq 0$$

$$= Cx^3 \quad = Cg(x)$$

We have: $C = 8, \quad k = 0, \quad g(x) = x^3$

$f(x)$ is $\Omega(x^3)$

Big-Theta

- $f(x)=O(g(x))$ only provides an upper bound in terms of $g(x)$ for $f(x)$.
- $f(x)=\Omega(g(x))$ only provides a lower bound in terms of $g(x)$ for $f(x)$.
- Big-Theta provides both an upper bound and a lower bound for a function in terms of a reference function $g(x)$

Big-Theta

Definition: Let f and g be functions from N (or R) to R. We say that "$f(x)$ is $\Theta(g(x))$" if "$f(x)$ is $O(g(x))$" and "$f(x)$ is $\Omega(g(x))$", i.e., there are positive constants C_1, C_2, and k such that

$$0 \leq C_1 g(x) \leq f(x) \leq C_2 g(x)$$

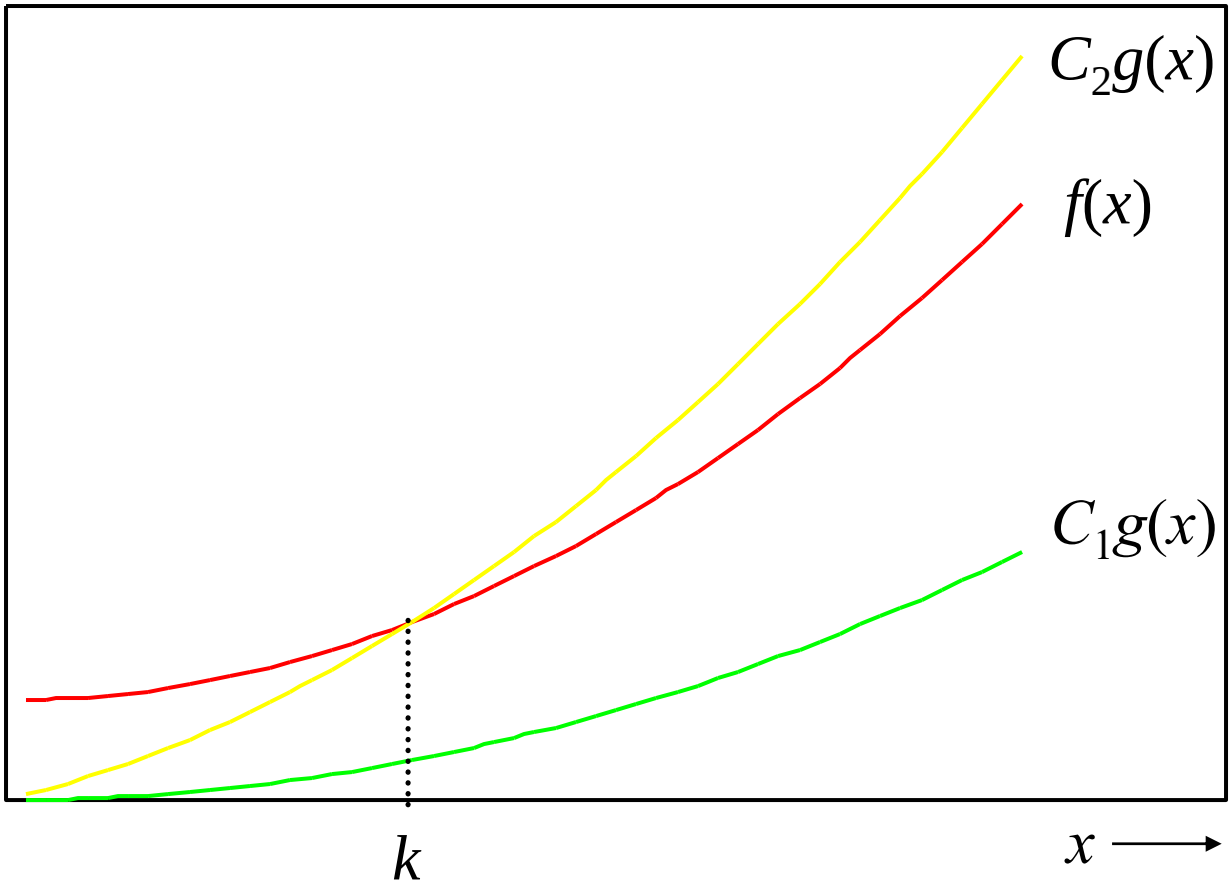
whenever $x \geq k$.

$f(x)$ is $\Theta(g(x))$ is read as:

"$f(x)$ is big-Theta of $g(x)$"

"$f(x)$ is of order $g(x)$"

Big-Theta



Show that $f(x) = 3x^2 + 8x\log x$ is $\Theta(x^2)$.

Solution:

$$f(x) = 3x^2 + 8x\log x$$
$$\leq 3x^2 + 8x^2 \qquad \text{For } x \geq 1$$
$$= 11x^2 \quad = C_2x^2 \quad = C_2g(x)$$

$\Big\}$ $f(x)$ is $O(x^2)$

$$f(x) = 3x^2 + 8x\log x$$
$$\geq 3x^2 \qquad \text{For } x \geq 1$$
$$= C_1x^2 \quad = C_1g(x)$$

$\Big\}$ $f(x)$ is $\Omega(x^2)$

We have: $C_1 = 3, C_2 = 11, k = 1, g(x) = x^2$

$f(x)$ is $\Theta(x^2)$

Exercises

- **P.228: 19, 36-37, 45, 53, 54, 71**

3.3 Complexity of Algorithms

- Means the cost of a program's execution
  - Running time, memory requirement, ...
- Doesn’t mean
  - The cost of creating the program i.e., # of statements, development time etc.
- In this context, programs with less complexity may require more development time.
- *Time Complexity*: Gives the approximate number of operations required to solve a problem of size *n*.
- *Space Complexity*: Gives the approximate amount of memory required to solve a problem of size *n*.
  - Often tied in with the particular data structures used to implement the algorithm

Different types of analysis

- Worst-case analysis
  - Maximum number of operations
  - Is a guarantee over all inputs of a given size
- Best-case analysis
  - Minimum number of operations
  - Not very practical
- Average-case analysis
  - Average number of operations assuming an input probability distribution
  - An average over all the possible inputs of a given size
  - Can be complicated

Running Time

- Simple statements (i.e., initialization of variables) have a complexity of $O(1)$.
- Loops have a complexity of $O(g(n)f(n))$, where $g(n)$ is upper bound on number of loop iterations and $f(n)$ is upper bound on the body of the loop.
  - If $g(n)$ and $f(n)$ are constant, then this is constant time.
- Conditional statements have a complexity of $O(\max(f(n),g(n)))$, where $f(n)$ is upper bound on then part and $g(n)$ is upper bound on else part.
- Blocks of statements with complexities $f_1(n)$, $f_2(n)$, …, $f_k(n)$, have complexity $O(f_1(n) + f_2(n) + ... + f_k(n))$.

Examples

Example: Describe the time complexity of the algorithm for finding the maximum element in a finite sequence.

```
procedure max(a_1, a_2, …., a_n: integers)
   max := a_1
   for i := 2 to n
       if max < a_i then max := a_i
   return max{max is the largest element}
```

Solution: Count the number of comparisons.

- The $max < a_i$ comparison is made $n - 2$ times.
- Each time i is incremented, a test is made to see if $i \leq n$.
- One last comparison determines that $i > n$.
- Exactly $2(n-1) + 1 = 2n - 1$ comparisons are made.

Hence, the time complexity of the algorithm is $\Theta(n)$.

Worst-Case Complexity of Linear Search

Example: Determine the time complexity of the linear search algorithm.

```
procedure linear search(x:integer,
          a_1, a_2, …,a_n: distinct integers)
i := 1
while (i ≤ n and x ≠ a_i)
    i := i + 1
if i ≤ n then location := i
else location := 0
return location{location is the subscript of the term that equals x, or is 0 if x is
    not found}
```

Solution: Count the number of comparisons.

- At each step two comparisons are made; $i \leq n$ and $x \neq a_i$.
- To end the loop, one comparison $i \leq n$ is made.
- After the loop, one more $i \leq n$ comparison is made.

If $x = a_i$, $2i + 1$ comparisons are used. If x is not on the list, $2n + 1$ comparisons are made and then an additional comparison is used to exit the loop. So, in the worst case $2n + 2$ comparisons are made. Hence, the complexity is $\Theta(n)$.

Average-Case Complexity of Linear Search

Example: Describe the average case performance of the linear search algorithm. (Although usually it is very difficult to determine average-case complexity, it is easy for linear search.)

Solution: Assume the element is in the list and that the possible positions are equally likely. By the argument on the previous slide, if $x = a_i$, the number of comparisons is $2i + 1$.

$$\frac{3+5+7+\ldots+(2n+1)}{n} = \frac{2(1+2+3+\ldots+n)+n}{n} = \frac{2[\frac{n(n+1)}{2}]}{n} + 1 = n + 2$$

Hence, the average-case complexity of linear search is $\Theta(n)$.

Worst-Case Complexity of Binary Search

Example: Describe the time complexity of binary search in terms of the number of comparisons used.

```
procedure binary search(x: integer, a_1,a_2,…, a_n: increasing integers)
 i := 1 {i is the left endpoint of interval}
 j := n {j is right endpoint of interval}
 while i < j
       m := ⌊(i + j)/2⌋
       if x > a_m then i := m + 1
       else j := m
 if x = a_i then location := i
 else location := 0
 return location{location is the subscript i of the term a_i  equal to x, or 0 if x is not found}
```

Solution: Assume (for simplicity) $n = 2^k$ elements. Note that $k = \log n$.

- Two comparisons are made at each stage; $i < j$, and $x > a_m$.
- At the first iteration the size of the list is 2^k and after the first iteration it is 2^{k-1}. Then 2^{k-2} and so on until the size of the list is $2^1 = 2$.
- At the last step, a comparison tells us that the size of the list is the size is $2^0 = 1$ and the element is compared with the single remaining element.
- Hence, at most $2k + 2 = 2 \log n + 2$ comparisons are made.
- Therefore, the time complexity is $\Theta(\log n)$, better than linear search.

Worst-Case Complexity of Bubble Sort

Example: What is the worst-case complexity of bubble sort in terms of the number of comparisons made?

procedure *bubblesort*($a_1,\ldots,a_n$: real numbers with $n \geq 2$)
  for i := 1 to $n-1$
    for j := 1 to $n-i$
      if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}
{$a_1,\ldots, a_n$ is now in increasing order}

$$(n-1)+(n-2)+\ldots+2+1 = \frac{n(n-1)}{2}$$

$$\frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Worst-Case Complexity of Insertion Sort

Example: What is the worst-case complexity of insertion sort in terms of the number of comparisons made?

Solution: The total number of comparisons are:

$$2 + 3 + \cdots + n = \frac{n(n-1)}{2} - 1$$

Therefore the complexity is $\Theta(n^2)$.

procedure *insertion sort*(a_1,…,a_n: real numbers with $n \geq 2$)
  for j := 2 to n
    i := 1
    while $a_j > a_i$
      $i := i + 1$
    $m := a_j$
    for k := 0 to $j - i - 1$
      $a_{j-k} := a_{j-k-1}$
    $a_i := m$

Complexity of Matrix Multiplication

Example: How many additions of integers and multiplications of integers are used by the matrix multiplication algorithm to multiply two $n \times n$ matrices.

Solution: There are n^2 entries in the product. Finding each entry requires n multiplications and $n - 1$ additions. Hence, n^3 multiplications and $n^2(n - 1)$ additions are used.

Hence, the complexity of matrix multiplication is $O(n^3)$.

Complexity of Boolean Product Algorithm

Example: How many bit operations are used to find $A \odot B$, where A and B are $n \times n$ zero-one matrices?

Solution: There are n^2 entries in the $A \odot B$. A total of n Ors and n ANDs are used to find each entry. Hence, each entry takes $2n$ bit operations. A total of $2n^3$ operations are used.

Therefore the complexity is $O(n^3)$

Commonly used Terminology for the Complexity of Algorithms

- O(1): constant complexity
- O(log n): logarithmic complexity
- O(n): linear complexity
- O(n log n): n log n complexity
- $O(n^b)$: polynomial complexity
- $O(b^n)$: exponential complexity(b>1)
- O(n!): factorial complexity

*Understanding the complexity of algorithm

- *Tractable Problem*: There exists a polynomial time algorithm to solve this problem. These problems are said to belong to the *Class P*.
- *Intractable Problem*: There does not exist a polynomial time algorithm to solve this problem
- *Unsolvable Problem* : No algorithm exists to solve this problem, e.g., halting problem.
- *Class NP*: Solution can be checked in polynomial time. But no polynomial time algorithm has been found for finding a solution to problems in this class. (Finding prime factors of a large number) (Sodoku) (The satisfiability problem)
- *NP Complete Class*: If you find a polynomial time algorithm for one member of the class, it can be used to solve all the problems in the class NP.

*Understanding the complexity of algorithm

- The *P versus NP problem* asks whether the class P = NP? Are there problems whose solutions can be checked in polynomial time, but can not be solved in polynomial time?
- If a polynomial time algorithm for any of the problems in the NP complete class were found, then that algorithm could be used to obtain a polynomial time algorithm for every problem in the NP complete class.
  - Satisfiability (in Section 1.3) is an NP complete problem.
- It is generally believed that P≠NP since no one has been able to find a polynomial time algorithm for any of the problems in the NP complete class.
- The problem of P versus NP remains one of the most famous unsolved problems in mathematics (including theoretical computer science). The Clay Mathematics Institute has offered a prize of $1,000,000 for a solution.

*Understanding the complexity of algorithm

- **旅行商问题（TSP）的判定版**：给定一张地图和一个预算 B，是否存在一条路径，经过所有城市且总花费 ≤ B？—— 求解很难，但验证一条给定路径是否满足条件很容易。
- **布尔可满足性问题（SAT）**：给定一个逻辑公式，是否存在一组变量赋值让公式为真？—— 求解困难，验证一个赋值是否有效很简单。

如果 P = NP 被证明，会对科技、经济、社会产生颠覆性影响：

- **密码学**：当前主流加密技术（如 RSA、椭圆曲线加密）的安全性，依赖于 “大整数分解是 NP 难问题”。如果 P=NP，就能找到多项式时间算法分解大整数，所有基于这类难题的密码体系都会失效。
- **人工智能与优化**：NP 完全问题在物流、调度、芯片设计等领域广泛存在，P=NP 意味着这些问题能被高效求解，极大推动工业优化和 AI 算法发展。
- **数学研究**：可以用计算机高效证明数学定理（因为定理证明属于 NP 类问题）。

Exercises

- **P.241: 12, 37**